

Research lý thuyết nền tảng

KHOA NGUYỄN TRẦN MINH và HỒ VĂN LỰC

July 2026

1 Curse of Dimensionality

1.1 Định nghĩa

Trong các bài toán học máy dựa trên khoảng cách (distance-based learning), đặc biệt là thuật toán k -Nearest Neighbors (KNN), việc xác định độ tương đồng giữa các điểm dữ liệu phụ thuộc trực tiếp vào khái niệm khoảng cách trong không gian đặc trưng (feature space). Tuy nhiên, khi số chiều d của không gian dữ liệu tăng lên, hiệu quả của các phép đo khoảng cách này suy giảm nghiêm trọng — hiện tượng này được gọi là **curse of dimensionality** (lời nguyền chiều dữ liệu).

Cụ thể, với một tập hợp điểm dữ liệu được sinh ngẫu nhiên trong không gian $\mathbb{R}^d$, khi $d \rightarrow \infty$, khoảng cách giữa một điểm truy vấn q và điểm gần nhất (nearest point) hội tụ về khoảng cách đến điểm xa nhất (farthest point), tức là:

$$\lim_{d \rightarrow \infty} \frac{\text{dist}_{\max}(d) - \text{dist}_{\min}(d)}{\text{dist}_{\min}(d)} \rightarrow 0 \quad (1)$$

Hiện tượng này được gọi là **distance concentration** (sự tập trung khoảng cách), lần đầu được chứng minh một cách hệ thống bởi Beyer et al. [1] trong công trình “*When is Nearest Neighbor Meaningful?*”. Kết quả này cho thấy, dưới một số điều kiện phân phối nhất định, khái niệm “hàng xóm gần nhất” dần mất đi ý nghĩa thống kê khi số chiều dữ liệu đủ lớn.

Hệ quả của curse of dimensionality đối với KNN bao gồm:

- **Mất tính phân biệt (loss of contrast):** khi số chiều tăng, tỉ lệ chênh lệch giữa khoảng cách gần nhất và xa nhất giảm dần về 0, khiến việc lựa chọn k hàng xóm gần nhất trở nên kém tin cậy.
- **Sparsity của dữ liệu (data sparsity):** để duy trì mật độ mẫu không đổi trong không gian d chiều, số lượng điểm dữ liệu cần thiết tăng theo cấp số mũ theo d , khiến dữ liệu thực tế luôn trở nên “thưa” khi d lớn.
- **Suy giảm ý nghĩa hình học của norm:** về mặt đại số tuyến tính, phương sai tương đối của khoảng cách Euclidean (norm L_2) giữa các điểm giảm dần so với giá trị kỳ vọng của khoảng cách khi d tăng, đây chính là cơ sở toán học của hiện tượng concentration of measure.

Vì vậy, việc hiểu rõ curse of dimensionality không chỉ là nền tảng lý thuyết quan trọng để lý giải sự suy giảm hiệu năng của KNN trong không gian chiều cao, mà còn là động lực trực tiếp cho việc áp dụng các kỹ thuật giảm chiều dữ liệu (dimensionality reduction) như PCA trong báo cáo này.

1.2 Chứng minh hiện tượng distance concentration

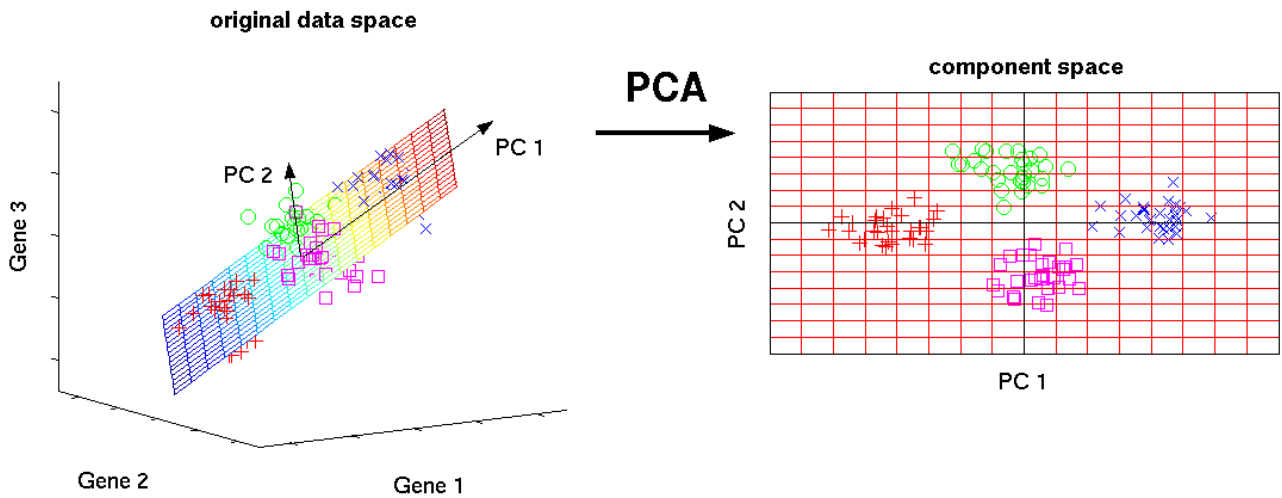
2 Principal Component Analysis (PCA)

2.1 Khái niệm

PCA là phương pháp biến đổi giúp giảm số lượng lớn các biến có tương quan với nhau thành tập ít các biến sao cho các biến mới tạo ra là tổ hợp tuyến tính của những biến cũ không có tương quan lẫn nhau.

Ví dụ hệ thống cảm biến: Giả sử chúng ta thu thập dữ liệu từ hệ thống điều hướng của một thiết bị bay (UAV) với $p = 20$ cảm biến khác nhau. Việc quan sát đồng thời 20 biểu đồ phân tán sẽ tạo ra $p(p-1)/2 = 190$ cặp đồ thị tương quan, khiến việc chẩn đoán trạng thái bay trở nên cực kỳ phức tạp. Tuy nhiên, PCA cho phép chúng ta chọn lọc ra một số ít các thành phần chính ($k \ll 20$) nhưng vẫn chứa đựng phần lớn thông tin về quỹ

đạo và độ ổn định của thiết bị.



Hình 1: Hình minh họa thuật toán PCA

Vấn đề cốt lõi của PCA là: Làm sao để giảm chiều mà không làm mất đi các đặc trưng quan trọng? Thuật toán giải quyết điều này bằng cách tìm kiếm những hướng (trục) mà tại đó dữ liệu có **phương sai (variance)** lớn nhất. Trong kỹ thuật, phương sai cao đồng nghĩa với việc chứa nhiều thông tin hữu ích, trong khi phương sai thấp thường được xem là nhiễu và không chứa nhiều giá trị trong việc phân loại.

2.2 Quy trình thực hiện thuật toán PCA chi tiết

Principal Component Analysis (PCA) được giới thiệu lần đầu bởi Pearson và phát triển thành công cụ phân tích hiện đại như trình bày trong [6]. Quy trình thực hiện gồm các bước sau:

Bước 1: Chuẩn hóa dữ liệu (Standardization)

Biểu thức:

$$z = \frac{\text{value} - \bar{x}}{\sigma}$$

Khái niệm: Đưa các biến về cùng một thang đo bằng cách trừ đi trung bình và chia cho độ lệch chuẩn.

Ứng dụng: Tránh tình trạng các biến có đơn vị hoặc biên độ lớn (ví dụ: giá nhà hàng tỷ đồng) lấn át các biến có biên độ nhỏ (ví dụ: số phòng ngủ), đảm bảo mọi thuộc tính đều đóng góp công bằng vào quá trình tìm thành phần chính.

Bước 2: Lập ma trận hiệp phương sai (Covariance Matrix)

Biểu thức:

$$S = \frac{1}{n-1} X^T X$$

Khái niệm: Một ma trận đối xứng chứa các giá trị phương sai trên đường chéo chính và hiệp phương sai ở các vị trí còn lại.

Ứng dụng: Ma trận này tổng hợp toàn bộ cấu trúc liên kết và sự phân tán của dữ liệu đa chiều. Nó là đối tượng đầu vào để thực hiện phép phân rã trị riêng, giúp tìm ra các hướng lan tỏa mạnh nhất của dữ liệu.

Bước 3: Tìm trị riêng và Vector riêng (Eigen-decomposition)

Biểu thức:

$$\det(S - \lambda I) = 0 \quad \text{và} \quad Sx = \lambda x$$

Khái niệm: Trị riêng (λ) biểu thị độ lớn của phương sai trên trục mới, còn Vector riêng (x) xác định hướng của trục đó.

Ứng dụng: Các Vector riêng ứng với Trị riêng lớn nhất chính là các Thành phần chính (Principal Components). Chúng xác định hệ trục tọa độ mới trực giao mà tại đó dữ liệu không còn tương quan với nhau.

Giải thích thêm: Ma trận hiệp phương sai S mô tả cách dữ liệu phân tán theo mọi hướng. Các eigenvector của S chính là những hướng đặc biệt mà theo đó phương sai của dữ liệu chỉ đơn thuần được “đo lường” (qua λ) mà không bị pha trộn với hướng khác. Đây là lý do PCA chọn eigenvector làm trục tọa độ mới — vì theo các trục này, dữ liệu không còn tương quan (uncorrelated) với nhau nữa.

Bước 4: Phép chiếu giảm chiều (Projection)

Biểu thức:

$$Y = XW$$

Khái niệm: Nhân ma trận dữ liệu gốc X với ma trận chuyển cơ sở W (gồm các Vector riêng đã chọn làm cột).

Ứng dụng: Đây là bước thực thi việc giảm chiều dữ liệu. Dữ liệu ban đầu được xoay và nén sang không gian mới. Kết quả là ta thu được một tập dữ liệu mới với số lượng biến ít hơn nhưng vẫn phản ánh được các xu hướng quan trọng nhất của dữ liệu ban đầu.

2.3 Code triển khai

Trong phần này, PCA được triển khai theo hai cách:

- **PCA được triển khai thủ công:** Tuân theo đúng các bước toán học đã trình bày ở phần định nghĩa: chuẩn hóa dữ liệu (standard scaling), xây dựng ma trận hiệp phương sai, phân rã trị riêng - vector riêng (eigen-decomposition), sau đó chiếu dữ liệu lên không gian con tạo bởi các thành phần chính có trị riêng lớn nhất.
- **PCA sử dụng thư viện scikit-learn:** Trong thực tế, các thư viện trong python cung cấp sẵn các hàm giúp ta giảm chiều dữ liệu bằng PCA mà không cần code thủ công lại từ đầu, điều này giúp code dễ đọc, dễ hiểu và không bị nặng lý thuyết.

2.3.1 Triển khai thủ công (Manual Implementation)

```
1 import numpy as np
2 import pandas as pd
3
4 def pca_manual(df: pd.DataFrame, n_components: int) -> pd.DataFrame:
5     """
6     Parameters
7     -----
8     df : pd.DataFrame
9         Dữ liệu đầu vào, các cột là biến số (feature), không bao gồm nhãn (label).
10    n_components : int
11        Số chiều cần giữ lại sau khi giảm chiều.
12
13    Returns
14    -----
15    pd.DataFrame
16        Dữ liệu sau khi giảm chiều, gồm n_components cột: PC1, PC2, ...
17    """
18
19    # ---- Bước 1: Chuẩn hóa dữ liệu ----
20    X = df.values.astype(float)
21    mean_vec = np.mean(X, axis=0)
22    std_vec = np.std(X, axis=0)
```

```

23 X_scaled = (X - mean_vec) / std_vec
24
25 # ---- Bước 2: Tính ma trận hiệp phương sai ----
26 n_samples = X_scaled.shape[0]
27 cov_matrix = (X_scaled.T @ X_scaled) / (n_samples - 1)
28
29 # ---- Bước 3: Phân ra trị riêng - vector riêng ----
30 eigen_values, eigen_vectors = np.linalg.eigh(cov_matrix)
31 # eigh trả về trị riêng theo thứ tự tăng dần -> cần đảo ngược lại
32
33 # ---- Bước 4: Sắp xếp giảm dần và chọn k thành phần đầu ----
34 sorted_idx = np.argsort(eigen_values)[::-1]
35 eigen_values_sorted = eigen_values[sorted_idx]
36 eigen_vectors_sorted = eigen_vectors[:, sorted_idx]
37
38 top_k_vectors = eigen_vectors_sorted[:, :n_components]
39
40 # ---- Bước 5: Chiếu dữ liệu lên không gian con mới ----
41 X_reduced = X_scaled @ top_k_vectors
42
43 columns = [f"PC{i+1}" for i in range(n_components)]
44 df_reduced = pd.DataFrame(X_reduced, columns=columns, index=df.index)
45
46 return df_reduced

```

Listing 1: Cài đặt PCA thủ công qua các bước toán học

Một số điểm cần lưu ý khi cài đặt thủ công:

- Dữ liệu cần được **scaling** (chuẩn hóa mean = 0, std = 1) trước khi tính ma trận hiệp phương sai, vì PCA nhạy cảm với thang đo (scale) giữa các biến — nếu không chuẩn hóa, các biến có phương sai lớn sẽ chi phối kết quả.
- Hàm `np.linalg.eigh` được dùng thay vì `np.linalg.eig` vì ma trận hiệp phương sai luôn là ma trận đối xứng (symmetric), `eigh` cho kết quả ổn định và nhanh hơn.
- Trị riêng trả về bởi `eigh` được sắp theo thứ tự tăng dần, nên cần đảo ngược (hoặc `argsort` giảm dần) trước khi chọn k thành phần chính đầu tiên.

2.3.2 Triển khai bằng thư viện scikit-learn

Trong thực tế, PCA thường được triển khai bằng thư viện `scikit-learn`:

```

1 from sklearn.preprocessing import StandardScaler
2 from sklearn.decomposition import PCA
3 import pandas as pd
4
5 def pca_sklearn(df: pd.DataFrame, n_components: int) -> pd.DataFrame:
6     """
7     Parameters
8     -----
9     df : pd.DataFrame
10         Dữ liệu đầu vào.
11     n_components : int
12         Số chiều cần giữ lại.
13
14     Returns
15     -----
16     pd.DataFrame
17         Dữ liệu sau khi giảm chiều, gồm n_components cột: PC1, PC2, ...
18     """
19
20     scaler = StandardScaler()
21     X_scaled = scaler.fit_transform(df.values)
22
23     pca = PCA(n_components=n_components)
24     X_reduced = pca.fit_transform(X_scaled)
25
26     columns = [f"PC{i+1}" for i in range(n_components)]
27     df_reduced = pd.DataFrame(X_reduced, columns=columns, index=df.index)
28
29     return df_reduced

```

Listing 2: Cài đặt PCA sử dụng thư viện có sẵn

Kết quả từ hai cách triển khai (thủ công và `sklearn`) là tương đương nhau về mặt giá trị tuyệt đối của phương sai được giữ lại, tuy nhiên dấu (sign) của các vector riêng có thể khác nhau — đây là hiện tượng bình thường trong eigen-decomposition (do v và $-v$ đều là nghiệm hợp lệ của phương trình trị riêng), không ảnh hưởng đến chất lượng giảm chiều.

Do đó, nhằm giữ tính ngắn gọn và dễ hiểu thì phần triển khai code chính sẽ triển khai bằng cách sử dụng thư viện.

2.4 Ưu và nhược điểm của PCA

Phần phân tích ưu và nhược điểm dưới đây được tổng hợp và trình bày lại dựa trên tài liệu tham khảo về PCA [4].

Ưu điểm

1. **Xử lý đa cộng tuyến (Multicollinearity Handling):** Tạo ra các biến mới không tương quan với nhau, giải quyết vấn đề khi các đặc trưng gốc có tương quan cao.
2. **Giảm nhiễu (Noise Reduction):** Loại bỏ các thành phần có phương sai thấp, giúp dữ liệu trở nên rõ ràng hơn.
3. **Nén dữ liệu (Data Compression):** Biểu diễn dữ liệu bằng số lượng thành phần ít hơn, giảm nhu cầu lưu trữ và tăng tốc độ xử lý.
4. **Phát hiện ngoại lệ (Outlier Detection):** Nhận diện các điểm dữ liệu bất thường thông qua mức độ lệch đáng kể của chúng trong không gian đã giảm chiều.

Nhược điểm

1. **Khó khăn trong diễn giải (Interpretation Challenges):** Các thành phần mới là tổ hợp tuyến tính của các biến gốc, gây khó khăn khi giải thích ý nghĩa thực tế.
2. **Nhạy cảm với việc chuẩn hóa dữ liệu (Data Scaling Sensitivity):** Yêu cầu dữ liệu phải được chuẩn hóa phù hợp trước khi áp dụng, nếu không kết quả có thể bị sai lệch.
3. **Mất mát thông tin (Information Loss):** Việc giảm số chiều có thể làm mất một số thông tin quan trọng nếu giữ lại quá ít thành phần chính.
4. **Giả định về tính tuyến tính (Assumption of Linearity):** PCA hoạt động hiệu quả nhất khi mối quan hệ giữa các biến là tuyến tính, và gặp khó khăn với dữ liệu phi tuyến.
5. **Độ phức tạp tính toán (Computational Complexity):** Có thể chậm và tốn nhiều tài nguyên tính toán trên các tập dữ liệu rất lớn.

3 Linear Discriminant Analysis (LDA)

3.1 Định nghĩa

Thuật toán Linear Discriminant Analysis (LDA) là một kỹ thuật giảm chiều có giám sát (supervised dimensionality reduction). Khác với PCA chỉ quan tâm đến việc giữ lại phương sai lớn nhất của dữ liệu, LDA tận dụng nhãn lớp (class labels) để tìm phép chiếu sao cho:

- Các điểm cùng lớp nằm gần nhau.
- Các lớp cách xa nhau nhất có thể.

Linear Discriminant Analysis (LDA) được Fisher giới thiệu lần đầu như một phương pháp phân loại tuyến tính [2]. Với sự phát triển của học máy và khai phá dữ liệu, LDA hiện được xem là một phương pháp giảm chiều có giám sát, đồng thời vẫn giữ vai trò là một bộ phân loại tuyến tính [5].

3.2 Quy trình thực hiện thuật toán LDA chi tiết

Bước 1: Tính vector trung bình của từng lớp (Class Means) Biểu thức:

$$\mu_c = \frac{1}{n_c} \sum_{i \in c} x_i, \quad \mu = \frac{1}{n} \sum_{i=1}^n x_i$$

Khái niệm: μ_c là vector trung bình của các điểm dữ liệu thuộc lớp c , còn μ là vector trung bình toàn cục của toàn bộ tập dữ liệu.

Ứng dụng: Đây là bước nền tảng để xác định "tâm" của từng lớp và tâm chung của dữ liệu, làm cơ sở đo lường mức độ phân tán trong lớp và giữa các lớp ở các bước tiếp theo.

Bước 2: Lập ma trận phân tán trong lớp và giữa các lớp (Within-class & Between-class Scatter Matrix) Biểu thức:

$$S_W = \sum_{c=1}^C \sum_{i \in c} (x_i - \mu_c)(x_i - \mu_c)^T, \quad S_B = \sum_{c=1}^C n_c (\mu_c - \mu)(\mu_c - \mu)^T$$

Khái niệm: S_W đo độ phân tán của các điểm dữ liệu xung quanh trung bình của lớp chúng thuộc về (biến thiên nội bộ lớp), còn S_B đo độ phân tán giữa trung bình các lớp so với trung bình toàn cục (biến thiên giữa các lớp).

Ứng dụng: Hai ma trận này tổng hợp toàn bộ thông tin về sự tách biệt (giữa các lớp) và độ đồng nhất (trong từng lớp) của dữ liệu, là đầu vào cốt lõi để xây dựng tiêu chuẩn tối ưu hóa của LDA.

Bước 3: Giải bài toán trị riêng tổng quát (Generalized Eigen-decomposition) Biểu thức:

$$S_W^{-1} S_B w = \lambda w$$

Khái niệm: Trị riêng (λ) biểu thị mức độ phân tách giữa các lớp mà trục chiếu tương ứng đạt được, còn Vector riêng (w) xác định hướng của trục chiếu đó.

Ứng dụng: Các Vector riêng ứng với Trị riêng lớn nhất chính là các Thành phần phân biệt (Linear Discriminants). Chúng xác định hệ trục tọa độ mới mà tại đó tỉ số giữa biến thiên giữa các lớp và biến thiên trong lớp (Fisher criterion) đạt giá trị lớn nhất.

Bước 4: Phép chiếu giảm chiều (Projection) Biểu thức:

$$Y = XW$$

Khái niệm: Nhân ma trận dữ liệu gốc X với ma trận chuyển cơ sở W (gồm các Vector riêng đã chọn làm cột, tối đa $C - 1$ cột với C là số lớp).

Ứng dụng: Đây là bước thực thi việc giảm chiều dữ liệu theo hướng có tính phân biệt (discriminative) cao nhất. Dữ liệu ban đầu được chiếu sang không gian mới với số chiều ít hơn, trong đó ranh giới giữa các lớp được thể hiện rõ ràng nhất có thể, phục vụ tốt cho các bài toán phân loại phía sau.

3.3 Code triển khai

Trong phần này, LDA được triển khai theo hai cách:

- **LDA được triển khai thủ công:** Tuân theo đúng các bước toán học đã trình bày ở phần định nghĩa: tính vector trung bình theo từng lớp và trung bình tổng thể, xây dựng ma trận tán xạ trong lớp (within-class scatter) và ma trận tán xạ giữa các lớp (between-class scatter), phân rã trị riêng - vector riêng của ma trận $S_W^{-1} S_B$, sau đó chiếu dữ liệu lên không gian con tạo bởi các vector riêng có trị riêng lớn nhất.
- **LDA sử dụng thư viện scikit-learn:** Trong thực tế, thư viện **scikit-learn** cung cấp sẵn hàm giúp ta giảm chiều dữ liệu bằng LDA mà không cần code thủ công lại từ đầu, điều này giúp code dễ đọc, dễ hiểu và không bị nặng lý thuyết.

3.3.1 Triển khai thủ công (Manual Implementation)

```
1 import numpy as np
2 import pandas as pd
3
4 def lda_manual(df: pd.DataFrame, y: np.ndarray, n_components: int) -> pd.DataFrame:
5     """
6     Parameters
7     -----
8     df : pd.DataFrame
9         Du lieu dau vao, cac cot la bien so (feature), khong bao gom nhan (label).
10    y : np.ndarray
11        Nhan lop tuong ung voi tung mau (sample).
12    n_components : int
13        So chieu can giu lai sau khi giam chieu (toi da la so_lop - 1).
14
15    Returns
16    -----
17    pd.DataFrame
18        Du lieu sau khi giam chieu, gom n_components cot: LD1, LD2, ...
19    """
20
21    X = df.values.astype(float)
22    n_features = X.shape[1]
23    classes = np.unique(y)
24
25    # ---- Buoc 1: Tinh vector trung binh tong the va trung binh tung lop ----
26    mean_overall = np.mean(X, axis=0)
27    mean_vectors = {c: np.mean(X[y == c], axis=0) for c in classes}
28
29    # ---- Buoc 2: Tinh ma tran tan xa trong lop (within-class scatter) ----
30    S_W = np.zeros((n_features, n_features))
31    for c in classes:
32        X_c = X[y == c]
33        mean_c = mean_vectors[c]
34        diff = X_c - mean_c
35        S_W += diff.T @ diff
36
37    # ---- Buoc 3: Tinh ma tran tan xa giua cac lop (between-class scatter) ----
38    S_B = np.zeros((n_features, n_features))
39    for c in classes:
40        n_c = X[y == c].shape[0]
41        mean_diff = (mean_vectors[c] - mean_overall).reshape(-1, 1)
42        S_B += n_c * (mean_diff @ mean_diff.T)
43
44    # ---- Buoc 4: Phan ra tri rieng - vector rieng cua S_W^{-1} S_B ----
45    eigen_values, eigen_vectors = np.linalg.eig(np.linalg.inv(S_W) @ S_B)
46    eigen_values = eigen_values.real
47    eigen_vectors = eigen_vectors.real
48
49    # ---- Buoc 5: Sap xep giam dan va chon k thanh phan dau ----
50    sorted_idx = np.argsort(eigen_values)[::-1]
51    eigen_vectors_sorted = eigen_vectors[:, sorted_idx]
52
53    top_k_vectors = eigen_vectors_sorted[:, :n_components]
54
55    # ---- Buoc 6: Chieu du lieu len khong gian con moi ----
56    X_reduced = X @ top_k_vectors
57
58    columns = [f"LD{i+1}" for i in range(n_components)]
59    df_reduced = pd.DataFrame(X_reduced, columns=columns, index=df.index)
60
61    return df_reduced
```

Listing 3: Cài đặt LDA thủ công qua các bước toán học

Một số điểm cần lưu ý khi cài đặt thủ công:

- Khác với PCA, LDA là phương pháp **có giám sát (supervised)**, do đó cần nhãn y để tính trung bình theo từng lớp và xây dựng các ma trận tán xạ.
- Số thành phần tối đa mà LDA có thể giữ lại là $C - 1$, với C là số lớp, vì ma trận S_B có hạng (rank) tối đa bằng $C - 1$.

- Ma trận S_W cần khả nghịch (invertible); trong trường hợp S_W suy biến (singular) — thường xảy ra khi số chiều lớn hơn số mẫu — cần áp dụng kỹ thuật giảm chiều trước (ví dụ PCA) hoặc dùng **pseudo-inverse** để tránh lỗi.
- Hàm `np.linalg.eig` được dùng thay vì `np.linalg.eigh` vì ma trận $S_W^{-1}S_B$ nói chung **không đối xứng**, nên trị riêng và vector riêng thu được có thể là số phức; phần ảo (imaginary part) trên lý thuyết bằng 0 nhưng do sai số tính toán có thể xuất hiện, nên cần lấy phần thực (`.real`) sau khi phân rã.

3.3.2 Triển khai bằng thư viện scikit-learn

Trong thực tế, LDA thường được triển khai bằng thư viện `scikit-learn`:

```

1 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
2 import pandas as pd
3
4 def lda_sklearn(df: pd.DataFrame, y, n_components: int) -> pd.DataFrame:
5     """
6     Parameters
7     -----
8     df : pd.DataFrame
9         Du lieu dau vao.
10    y : array-like
11        Nhan lop tuong ung voi tung mau.
12    n_components : int
13        So chieu can giu lai (toi da la so_lop - 1).
14
15    Returns
16    -----
17    pd.DataFrame
18        Du lieu sau khi giam chieu, gom n_components cot: LD1, LD2, ...
19    """
20
21    lda = LinearDiscriminantAnalysis(n_components=n_components)
22    X_reduced = lda.fit_transform(df.values, y)
23
24    columns = [f"LD{i+1}" for i in range(n_components)]
25    df_reduced = pd.DataFrame(X_reduced, columns=columns, index=df.index)
26
27    return df_reduced

```

Listing 4: Cài đặt LDA sử dụng thư viện có sẵn

Kết quả từ hai cách triển khai (thủ công và `sklearn`) tương đương nhau về mặt không gian con được tìm ra để phân tách các lớp, tuy nhiên `scikit-learn` sử dụng các phương pháp phân rã số học ổn định hơn (ví dụ SVD thay vì tính trực tiếp $S_W^{-1}S_B$), giúp tránh các vấn đề số học khi S_W gần suy biến — đây là điểm khác biệt về mặt cài đặt chứ không ảnh hưởng đến bản chất thuật toán.

Do đó, nhằm giữ tính ngắn gọn, dễ hiểu và ổn định về mặt số học thì phần triển khai code chính sẽ triển khai bằng cách sử dụng thư viện.

3.4 Ưu và nhược điểm của LDA

Phần phân tích ưu và nhược điểm dưới đây được tổng hợp và trình bày lại dựa trên tài liệu tham khảo về LDA [3].

Ưu điểm

1. **Đơn giản và hiệu quả về mặt tính toán:** LDA có cấu trúc toán học đơn giản, dễ triển khai và tốc độ xử lý nhanh.
2. **Hoạt động tốt khi số lượng đặc trưng lớn hơn nhiều so với số lượng mẫu huấn luyện:** LDA vẫn cho kết quả ổn định trong các trường hợp dữ liệu có chiều cao nhưng ít mẫu.
3. **Có thể xử lý đa cộng tuyến (Multicollinearity):** LDA có khả năng xử lý tốt các đặc trưng có tương quan cao với nhau.

Nhược điểm

1. **Giả định phân phối Gaussian:** LDA giả định dữ liệu tuân theo phân phối chuẩn (Gaussian), điều này không phải lúc nào cũng đúng trong thực tế.

2. **Giả định ma trận hiệp phương sai bằng nhau giữa các lớp:** Giả định này có thể không phù hợp với nhiều tập dữ liệu thực tế, nơi các lớp có phương sai khác nhau.
3. **Giả định tính khả phân tuyến tính (Linear Separability):** LDA giả định ranh giới phân chia giữa các lớp là tuyến tính, điều này không phải lúc nào cũng đúng.
4. **Có thể hoạt động kém trong không gian đặc trưng nhiều chiều:** Hiệu suất của LDA có thể giảm khi làm việc với dữ liệu có số chiều rất cao.

4 Giải thích thêm một số vấn đề

4.1 Vì sao phải tính $S_W^{-1}S_B$ trong thuật toán LDA?

Đây mới là ý tưởng cốt lõi của LDA.

Ta muốn tìm hướng w sao cho:

- khoảng cách giữa các lớp theo hướng đó là lớn nhất,
- đồng thời độ phân tán trong từng lớp theo hướng đó là nhỏ nhất.

Xuất phát từ trường hợp đơn giản: 2 lớp

Xét trường hợp chỉ có 2 lớp. Sau khi chiếu dữ liệu lên một hướng w , mỗi lớp trở thành một phân phối một chiều với trung bình (mean) và phương sai (variance) riêng:

$$\mu_i = \frac{1}{n_i} \sum_{x \in c_i} w^T x, \quad s_i^2 = \sum_{x \in c_i} (w^T x - \mu_i)^2, \quad i = 1, 2$$

Hai tiêu chí đặt ra ở trên tương ứng chính xác với:

$$\text{Khoảng cách giữa 2 lớp lớn nhất} \implies \max(\mu_1 - \mu_2)^2$$

$$\text{Độ phân tán trong từng lớp nhỏ nhất} \implies \min(s_1^2 + s_2^2)$$

Kết hợp hai mục tiêu trái ngược nhau (một cái muốn max, một cái muốn min) một cách tự nhiên bằng một tỉ số:

$$J(w) = \frac{(\mu_1 - \mu_2)^2}{s_1^2 + s_2^2}$$

Tối đa hóa $J(w)$ chính là đồng thời kéo hai trung bình lớp ra xa nhau (tử số lớn) và nén độ phân tán trong từng lớp lại (mẫu số nhỏ) — đúng như 2 tiêu chí ban đầu.

Tổng quát hóa cho nhiều lớp

Khi có $C > 2$ lớp, khái niệm "khoảng cách giữa các lớp" $((\mu_1 - \mu_2)^2)$ và "độ phân tán trong lớp" $(s_1^2 + s_2^2)$ được tổng quát hóa thành hai ma trận:

- **Độ phân tán giữa các lớp** theo hướng w (tổng quát của $(\mu_1 - \mu_2)^2$):

$$w^T S_B w$$

- **Độ phân tán trong các lớp** theo hướng w (tổng quát của $s_1^2 + s_2^2$):

$$w^T S_W w$$

với S_B (between-class scatter) và S_W (within-class scatter) đã định nghĩa ở phần trước, đóng vai trò như "phương sai giữa các trung bình lớp" và "tổng phương sai trong từng lớp" ở dạng ma trận nhiều chiều.

Tiêu chuẩn Fisher (Fisher criterion)

LDA tối đa hóa tỉ số:

$$J(w) = \frac{w^T S_B w}{w^T S_W w}$$

đây chính là phiên bản nhiều chiều của tỉ số $\frac{(\mu_1 - \mu_2)^2}{s_1^2 + s_2^2}$ ở trên.

Vì sao dẫn đến $S_W^{-1}S_B$?

Giải bài toán tối ưu này (lấy đạo hàm $J(w)$ theo w và cho bằng 0) dẫn đến phương trình trị riêng tổng quát:

$$S_B w = \lambda S_W w$$

Nhân cả hai vế cho S_W^{-1} (giả sử S_W khả nghịch):

$$S_W^{-1} S_B w = \lambda w$$

Đây chính là bài toán trị riêng thông thường của ma trận $S_W^{-1} S_B$. Vector riêng w ứng với trị riêng λ lớn nhất chính là hướng chiếu tối ưu — vừa tối đa hóa khoảng cách giữa các lớp (tử số), vừa tối thiểu hóa độ phân tán trong từng lớp (mẫu số), đúng như mục tiêu ban đầu của LDA.

Tài liệu

- [1] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is “nearest neighbor” meaningful? In *International Conference on Database Theory (ICDT)*, 1999.
- [2] R. A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.
- [3] GeeksforGeeks. Linear discriminant analysis in machine learning. <https://www.geeksforgeeks.org/machine-learning/ml-linear-discriminant-analysis/>, 2025. Last updated: 13 Sep. 2025. Accessed: 16 Jul. 2026.
- [4] GeeksforGeeks. Principal component analysis (pca). <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>, 2026. Last updated: 10 Jul. 2026. Accessed: 16 Jul. 2026.
- [5] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2nd edition, 2009.
- [6] Ian T. Jolliffe. *Principal Component Analysis*. Springer, 2nd edition, 2002.